触点检测

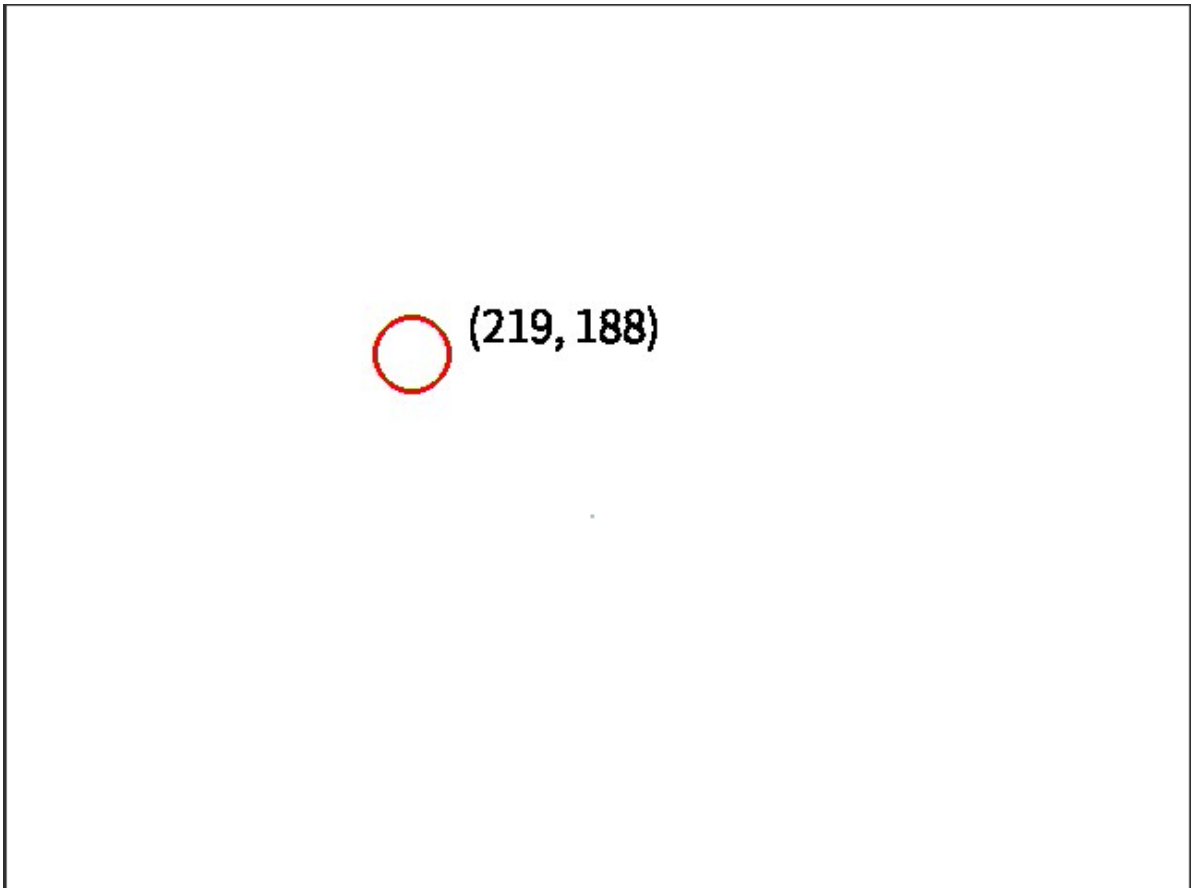
触点检测

- 例程效果
- 代码讲解
  - 1. 导入必要的模块
  - 2. 初始化触摸传感器和屏幕分辨率
  - 3. 定义触摸点显示函数
    - 3.1 创建并初始化背景图像
    - 3.2 初始化显示器和媒体管理器
  - 4. 触摸检测与处理逻辑
    - 4.1 定义触摸状态变量
    - 4.2 主循环：读取并处理触摸数据
      - 4.2.1 处理触摸按下事件
      - 4.2.2 处理触摸抬起事件
    - 4.3 更新显示
  - 5. 异常处理与资源释放
  - 6. 主程序入口

例程效果

本节例程源码文件名为 [1.show_touch_info.py]

连接IDE，在IDE中运行代码，点击屏幕上的任意位置，会看到屏幕中出现触摸点的信息



代码讲解

本节将详细解析 `1.show_touch_info.py` 代码的功能和实现逻辑，帮助用户理解如何通过触摸传感器捕获触点信息并在屏幕上显示。

1. 导入必要的模块

```
import time, os, urandom, sys
from media.display import *
from media.media import *
from machine import TOUCH
```

- `time`, `os`, `urandom`, `sys`：这些是系统相关的模块，用于时间控制、操作系统功能、随机数生成以及系统操作。
- `media.display` 和 `media.media`：用于屏幕显示和媒体资源管理。
- `machine.TOUCH`：用于初始化和操作触摸传感器。

2. 初始化触摸传感器和屏幕分辨率

```
tp = TOUCH(0)
DISPLAY_WIDTH = 640
DISPLAY_HEIGHT = 480
```

- `TOUCH(0)`：初始化触摸传感器，`tp` 用于后续读取触摸数据。
- `DISPLAY_WIDTH` 和 `DISPLAY_HEIGHT`：定义屏幕分辨率常量，分别为 640x480。

3. 定义触摸点显示函数

```
def touch_point_display():
    print("Touch point display test")
```

- `touch_point_display()` 是主函数，负责处理触摸点的检测和显示。

3.1 创建并初始化背景图像

```
img = image.Image(DISPLAY_WIDTH, DISPLAY_HEIGHT, image.RGB888)

def clear_and_reset_background():
    """Clear the image and reset to white background"""
    img.clear()
    img.draw_rectangle(0, 0, DISPLAY_WIDTH, DISPLAY_HEIGHT, color=(255,255,255),
fill=True)

clear_and_reset_background()
```

- 创建一个 RGB888 格式的图像对象 `img`，大小与屏幕分辨率一致。
- 定义 `clear_and_reset_background()` 函数，用于清除图像内容并重置为白色背景。
- 初始化时调用该函数，确保屏幕初始状态为白色。

3.2 初始化显示器和媒体管理器

```
Display.init(Display.ST7701, width = DISPLAY_WIDTH, height = DISPLAY_HEIGHT,
to_ide = True)
MediaManager.init()
```

- 使用 `ST7701` 驱动初始化显示器，设置分辨率为 640x480，并启用 IDE 输出（`to_ide = True`）。
- 初始化媒体管理器，用于管理显示相关的资源。

4. 触摸检测与处理逻辑

4.1 定义触摸状态变量

```
current_touch_active = False
last_touch_down_time = 0
```

- `current_touch_active`：布尔值，记录当前是否处于触摸状态。
- `last_touch_down_time`：记录上次触摸按下事件的时间。

4.2 主循环：读取并处理触摸数据

```
while True:
    os.exitpoint()
    point = tp.read(1)
```

- 使用 `tp.read(1)` 读取触摸数据，`point` 存储触摸点信息。

4.2.1 处理触摸按下事件

```
if len(point):
    pt = point[0]
    if pt.event == TOUCH.EVENT_DOWN:
        clear_and_reset_background()
        circle_radius = 20
        img.draw_circle(pt.x, pt.y, circle_radius, color=(255, 0, 0),
thickness=3)
        coord_text = f"({pt.x}, {pt.y})"
        text_x = pt.x + 30
        text_y = pt.y - 30
        if text_x + 100 > DISPLAY_WIDTH:
            text_x = pt.x - 100
        if text_y < 20:
            text_y = pt.y + 40
        img.draw_string_advanced(text_x, text_y, 24, coord_text, color=(0, 0,
0))
        current_touch_active = True
        last_touch_down_time = time.time()
        print(f"Touch detected at: {coord_text}")
```

- 当检测到触摸按下事件（`TOUCH.EVENT_DOWN`）时：
    - 清除之前的图像并重置背景为白色。
    - 在触摸点位置绘制一个半径为 20 的红色圆圈。
    - 计算并显示触摸点的坐标文字，文字位置根据触摸点位置进行调整，确保不超出屏幕边界。

- 更新触摸状态变量 `current_touch_active` 为 `True`，并记录按下时间。
- 在控制台打印触摸点坐标信息。

4.2.2 处理触摸抬起事件

```
elif pt.event == TOUCH.EVENT_UP:
    current_touch_active = False
```

- 当检测到触摸抬起事件（`TOUCH.EVENT_UP`）时，将 `current_touch_active` 设置为 `False`，表示触摸已结束。

4.3 更新显示

```
Display.show_image(img)
time.sleep(0.05)
```

- 使用 `Display.show_image(img)` 更新屏幕显示当前图像。
- `time.sleep(0.05)` 每 50 毫秒更新一次，避免过快刷新导致性能问题。

5. 异常处理与资源释放

```
except KeyboardInterrupt as e:
    print("User stop: ", e)
except BaseException as e:
    print(f"Exception {e}")

Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

- 捕获 `KeyboardInterrupt`（用户中断）和 `BaseException`（其他异常），并打印相关信息。
- 调用 `Display.deinit()` 反初始化显示器。
- 调用 `MediaManager.deinit()` 释放媒体资源。

6. 主程序入口

```
if __name__ == "__main__":
    os.exitpoint(os.EXITPOINT_ENABLE)
    touch_point_display()
```

- 启用退出点（`EXITPOINT_ENABLE`），并调用 `touch_point_display()` 函数启动触摸点显示测试。